ARMY INSTITUTE OF TECHNOLOGY PUNE | DEPARTMENT OF COMPUTER ENGINEERING

Setu Raksha

National Bridge Infrastructure Safety & Intelligence Grid

Antigravity Master Build Prompt Pack

Version 1.0 | 19 October 2026 | Dev Chahar (Comp-A) | Based on: Problem Identification and Modification Report

This pack contains everything you need to build Setu Raksha from an empty folder to a professional, demo-ready platform using Google Antigravity (or any agentic IDE). It has **one master prompt** that sets permanent project rules, **nine phase prompts** that you run one at a time, and **appendices** (26-factor inspection model, database entities, API surface, risk-score maths, acceptance checklist) that the agent must treat as specification.

Part	What it contains	How you use it
Part A	Master project prompt: mission, context from your report, roles, lifecycle, non-negotiable rules, stack, repo layout	Paste once into Antigravity workspace Rules (always-on context)
Part B	Phase 0 to Phase 8 mission prompts with build tasks and acceptance criteria	Paste one phase at a time as a new agent mission in the Manager view
Appendix A to E	Factor catalogue, schema, API table, SVI maths, definition-of-done, recovery prompts	Save into `docs/spec/` in the repo so the agent can read them

How to run this in Antigravity

1. Create an empty folder `setu-raksha/`, run `git init`, open it as a workspace in Antigravity, and save Appendices A to E as markdown files under `docs/spec/`.
2. Paste **Part A** into the workspace Rules (the always-on instructions location in your Antigravity version) so every agent session inherits it.
3. Open the Manager view, start a new agent with the **Phase 0** prompt in Planning mode, and read the Implementation Plan artifact **before** approving. Edit the plan if it drifts from Part A.
4. When the agent finishes, review its Task List and Walkthrough artifacts, run the acceptance commands yourself, commit, then tag `phase-N-done`. Only then start the next phase.
5. Run phases in order. Phases 2 (i18n) and 4 (simulator) can run in parallel agents after Phase 1 because they touch different directories; every other phase depends on the one before.
6. Use your strongest available model for Phases 0, 1 and 5 (architecture, schema, risk engine). A faster model is fine for UI polish and documentation.

FRAMING NOTES FOR YOUR FACULTY REVIEW (read before you submit)

- **The 90% figure is a design target, not a measured result.** Phase 8 builds a simulation-based evaluation protocol so you can report a defensible number (for example, the share of simulated hazard scenarios where a Critical alert was raised early enough) instead of claiming real-world prevention.
- **Percentages in your report (floods 45%, earthquakes 25%) are labelled illustrative** in the report itself. Cite them as indicative trends, or replace them with figures from FHWA HEC-18, NDMA or published failure studies.
- **Sensor streams and the starter bridge dataset are simulated.** The prompts force the product to label them SIMULATED everywhere, and to state that the system supports engineering judgement and does not replace inspection (your report, section 11.1).

Part A. Master Project Prompt (paste into workspace Rules)

Everything inside the blue boxes is prompt text to copy. Text outside the boxes is guidance for you.

A.1 Role and mission

You lead a multi-agent engineering squad: solution architect, backend engineer, frontend engineer, data/ML engineer, QA engineer, DevOps engineer and technical writer. Your mission is to build **Setu Raksha, the National Bridge Infrastructure Safety & Intelligence Grid**, from an empty repository into a demo-ready, professionally engineered platform whose code quality, documentation and security posture would let a state public-works department pilot it after review.

Never ship toy code, hidden shortcuts, hard-coded demo values presented as real logic, or unexplained dependencies. If a requirement is ambiguous, state your assumption in the plan artifact and continue; ask me only when the choice is irreversible or changes the architecture.

A.2 Problem context (from our approved report)

- Bridges and flyovers collapse in natural disasters (floods and scour, earthquakes, landslides and cloudbursts, cyclones) when an **extreme load meets a hidden weakness**: poor design, weak construction, ageing, no maintenance. The disaster is the trigger; the hidden weakness is the cause.
- The problem has a **physical side** (the structure is weak) and an **information side** (authorities do not know which bridge is at risk or when to close it).
- **Research gap:** there is no low-cost, simple, integrated system that combines inspection records, rainfall/flood data and sensor readings into one easy risk level per bridge and sends timely, bridge-specific alerts to local authorities.
- **Product position:** decision support. Setu Raksha helps engineers decide which bridge to inspect, restrict or close first. It never replaces detailed structural analysis or an engineer's judgement, and every screen that shows a risk level must say so.
- **Design target:** reduce preventable disaster-related bridge failures by up to 90% through earlier detection and action. This is a target to be tested by simulation in Phase 8. Never state it as an achieved or real-world result anywhere in code, UI or docs.

A.3 Users and roles

Role	Who	Core capabilities
Citizen (public)	Anyone, no login	Browse map, view a bridge's age, build-quality summary and live health tier; report a concern
Project Officer	Field engineer assigned to bridges	Monthly 26-factor audit, evidence upload, drafts, receives reminders and alerts
Division Engineer	Reviewer for a jurisdiction	Approve or return audits, acknowledge alerts, log actions (close, divert, inspect), receives escalations
State Analyst / Admin	Department headquarters	Analytics, heatmaps, compliance, thresholds, user and bridge management
System Admin	Platform operator	Configuration, audit logs, health, notification providers

A.4 Lifecycle coverage (build all seven)

- **L1 Pre-construction and design:** IRC loading class, seismic zone (IS 1893), design life, foundation type and depth, design high flood level, design scour depth, span configuration.
- **L2 Construction and quality audits:** material grade logs, cube/test results, initial load-test records, contractor history, quality grade, defect liability period.
- **L3 Inauguration and live monitoring:** streams for river water level, pier tilt, vibration and rainfall. Simulated first; the ingestion contract must be device-ready.
- **L4 Routine monthly audits:** a scheduler sends reminders to the assigned officer exactly 7 days before month-end, with escalation if the audit is missed.
- **L5 Officer portal and 26-factor ingestion:** structured form across structural, hydraulic, seismic, load, and age/environment variables (Appendix A).

- **L6 AI predictive risk engine:** Structural Vulnerability Index (SVI), dynamic risk with live hazards, Remaining Useful Life (RUL), explanations and prescriptive remediation (riprap, bearing replacement, and so on).
- **L7 Public transparency portal:** citizen view of build quality, age and live health tier with safe redaction.

A.5 Non-negotiable engineering rules

- **Trilingual from day one:** English, Hindi (हिन्दी), Marathi (मराठी). No hard-coded user-facing string anywhere. Switching language must never lose form state or data.
- **Language-neutral data:** store codes, numbers, ISO-8601 UTC timestamps and units in SI. Translate only at render time.
- **Typed everywhere:** TypeScript strict, Python type hints with mypy, Pydantic v2 models, OpenAPI as the single contract. Generate the frontend client from OpenAPI.
- **Tests are part of every task:** a task is not done until unit and integration tests pass. Targets: backend coverage 80% or higher, risk engine 95% or higher, Playwright end-to-end for every role.
- **Security by default:** argon2id password hashing, short-lived JWT in httpOnly cookies, RBAC plus jurisdiction scoping on every query, input validation, rate limits on public routes, append-only audit log, no secrets in git, upload type and size checks.
- **Safety-critical text is never freely generated.** Remediation advice comes from a versioned, reviewable catalogue (Appendix D). Cite standards by name only (IRC:6, IRC:SP:35, IS 1893, FHWA HEC-18). Never invent clause numbers or numeric code limits; put thresholds in config marked `example, to be validated by a civil engineer`.
- **Honest labelling:** every simulated record and stream carries `source=simulated` and is visibly labelled. Every risk output shows model version, timestamp and an advisory disclaimer.
- **Accessibility and reach:** WCAG 2.1 AA, never colour alone for risk tier (icon plus text), mobile-first, usable on slow networks.
- **Engineering hygiene:** Conventional Commits, small PRs, ADRs for decisions, 12-factor config, structured JSON logs with request IDs, migrations reversible, idempotent jobs.

A.6 Fixed technology stack

Layer	Choice	Notes
Web app	Next.js 15 (App Router), TypeScript strict, Tailwind CSS, Radix/shadcn primitives	next-intl for i18n, react-hook-form + Zod, TanStack Query, Zustand, Recharts or uPlot, react-leaflet + OpenStreetMap tiles with attribution
API	Python 3.12, FastAPI, SQLAlchemy 2.0, Alembic, Pydantic v2, uv for dependencies	Async endpoints, OpenAPI 3.1, SSE for live streams
Database	PostgreSQL 16 + PostGIS	Relational core; time-series table partitioned monthly (TimescaleDB optional); JSONB only for flexible payload snapshots
Jobs and scheduling	Celery + Redis with Celery Beat	Idempotent tasks, retries with backoff, Asia/Kolkata timezone
Notifications	Adapters: Email (SMTP, Mailpit locally), SMS (provider interface, mock locally), In-app (SSE)	Production SMS in India needs provider DLT template registration; document it
Files	MinIO (S3-compatible) with presigned URLs	Inspection photos and documents
ML	scikit-learn, XGBoost (GPU on the RTX laptop), SHAP, Hypothesis for property tests	Model card, version registry in `ml/registry/`; rule engine is the safety floor
Platform	Docker Compose, GitHub Actions, pre-commit, ruff, mypy, eslint, Playwright, k6, OWASP ZAP baseline	One command starts everything: `make up`

A.7 Repository layout

```
setu-raksha/
  apps/web/            Next.js app (public, officer, engineer, admin)
  apps/api/            FastAPI app: app/{core,db,auth,bridges,audits,telemetry,
                       risk,notify,public,admin}, alembic/, tests/
  services/simulator/  telemetry + scenario simulator
  services/worker/     Celery worker + beat entrypoints
  packages/shared/     JSON schemas, generated API client, i18n glossary
```

```
ml/                 data generator, training, calibration, registry, model card
infra/              docker-compose, Dockerfiles, CI, k6, ZAP config
docs/               spec/, adr/, architecture/, runbooks/, user-guides/{en,hi,mr}
Makefile  README.md  .env.example  CHANGELOG.md
```

A.8 Working agreement with the agent

- For every mission: produce an **Implementation Plan** first, list assumptions, then a **Task List**, then build, then a **Walkthrough** with commands to verify and screenshots or recordings of UI flows.
- Work in small commits on a branch per phase. Run lint, type-check and tests before declaring done. Show failing output honestly; never mark a task complete with failing tests or skipped checks.
- Read `docs/spec/` before writing code. If code and spec disagree, stop and tell me, do not silently change the spec.
- Do not add dependencies without a one-line justification in the plan. Prefer boring, well-maintained libraries. Pin versions.
- At the end of each phase update README, CHANGELOG, ADRs and the data dictionary.

Part B. Phase Prompts (run one at a time)

Each phase prompt is self-contained. Paste it as a new mission, let the agent plan, review the plan, then approve. Commit and tag when its acceptance criteria pass.

Phase 0. Foundation and developer platform

MISSION

Bootstrap the Setu Raksha monorepo and developer platform exactly as in Part A. No product features yet.

BUILD

- Create the repository layout from A.7 with pnpm workspaces for web and shared packages and uv for the API, simulator and worker.
- Write `docker-compose.yml` with postgres+postgis, redis, minio, mailpit, api, worker, beat, simulator and web, each with healthchecks, named volumes and `.env.example`. Add `Makefile` targets: `up, down, test, lint, seed, migrate, logs`.
- FastAPI skeleton: `/health`, `/ready`, pydantic-settings config, structured JSON logging with request IDs, global error handler returning `{code, params, request_id}` (never raw messages).
- Next.js skeleton: strict TypeScript, Tailwind design tokens (institutional navy/teal palette, risk-tier colours with icons, 4/8 px spacing scale, type scale), accessible app shell with header, language switcher placeholder and footer disclaimer.
- CI in GitHub Actions: ruff, mypy, eslint, tsc, unit tests, container build, dependency audit, secret scan. Add pre-commit hooks and Conventional Commits check.
- Write ADR-0001 (stack), ADR-0002 (i18n strategy), ADR-0003 (risk engine: rules first, ML as advisor) and a C4 context diagram in Mermaid.

ACCEPTANCE

- From a clean clone, `make up` brings every service to healthy in under 5 minutes; `make test` and CI are green; README quickstart verified by running it.

Phase 1. Data model, authentication and lifecycle L1 and L2

MISSION

Implement the relational model from Appendix B, secure authentication with RBAC, the bridge registry, and lifecycle phases L1 (design metadata) and L2 (construction and quality audits).

BUILD

- SQLAlchemy 2.0 models and reversible Alembic migrations for every entity in Appendix B: UUID keys, timestamps, soft delete where stated, PostGIS point geometry for bridge location, lookup tables for enums, indexes for the access patterns in Appendix C.
- Auth: admin-created users, login, refresh, logout, argon2id, httpOnly cookies, CSRF protection, role and **jurisdiction scoping** enforced in a reusable query dependency so that no endpoint can return another jurisdiction's private data.
- Bridge registry: CRUD, filters (district, river, tier, age band), full-text search, officer assignment history. L1 fields validated: seismic zone II to V, design life, foundation type and depth, design HFL, design scour depth, IRC loading class, span configuration.
- L2: material grade logs (concrete grade, steel grade, test results with pass/fail against configured minimums), initial load-test records, contractor history with quality grade, defect liability dates.
- Append-only audit log for every create, update and delete, with actor, before/after diff and request ID.
- Seed command creating about 60 clearly labelled **synthetic** bridges across Maharashtra river basins (Mula-Mutha, Bhima, Krishna, Godavari, Konkan rivers) with plausible ages, spans and zones, 8 officers, 3 division engineers and 1 admin.

ACCEPTANCE

- OpenAPI documents every route with examples. Negative RBAC tests prove officers cannot read or write outside their assignments. Migrations upgrade and downgrade cleanly. Coverage 80% or higher on the new modules.

Phase 2. Trilingual platform (English, Hindi, Marathi)

MISSION

Build the localisation layer used by every later feature: public viewer, officer portal, inspection forms, remediation text and the notification engine. Switching language must be instant and must never lose form state.

FRONTEND

- Use next-intl with locale-prefixed routes `/en, /hi, /mr`, message files per locale split by namespace (`common, public, officer, inspection, risk, notifications, errors, enums`), ICU MessageFormat for plurals and parameters, and typed message keys.
- **State-preserving switch:** form values live in a locale-independent store (react-hook-form mirrored to Zustand and autosaved to IndexedDB every 2 seconds). The switcher calls `router.replace` with the same path, query and hash, no scroll jump and no full reload. The layout above the locale segment stays mounted. Validation errors are stored as codes and re-rendered in the new language.
- Fonts: Noto Sans and Noto Sans Devanagari through `next/font`, with line-height tuned for matras. Add a pseudo-locale (`en-XA`) and test the longest Marathi strings for overflow on 360 px screens.
- Formats via `Intl`: dates DD/MM/YYYY in IST, Latin digits by default with a user preference for Devanagari digits, units rendered by one formatter.
- CI guard: fail the build on missing or extra keys across locales, untranslated values that equal English (with an allow-list), and hard-coded strings in JSX (custom eslint rule).

BACKEND

- Locale from user preference, then `Accept-Language`. Errors return code plus params. Email and SMS templates in `templates/{email,sms}/{en,hi,mr}` (Jinja2). Handle SMS Unicode segmentation (Devanagari uses UCS-2, 70 characters per segment) and warn if a message exceeds 3 segments.
- Remediation and recommendation text comes from the catalogue with ICU parameters, authored in all three languages with a `reviewed_by` field. Do not machine-translate safety text at runtime.
- Free-text officer remarks are stored as `{text, lang}` and shown in the original language with a badge. No automatic translation in v1; record this in an ADR.
- Create `packages/shared/i18n-glossary.yaml` for engineering terms (examples: bridge = पुल in Hindi, पूल in Marathi; risk = जोखिम in Hindi, जोखीम in Marathi; scour, riprap, bearing, pier, abutment) and mark every term **pending review by a Hindi and Marathi speaker with civil engineering knowledge**.

ACCEPTANCE

- Playwright test: fill 10 fields and attach a photo in English, trigger a validation error, switch English to Hindi to Marathi to English; assert zero value loss, error text in the active language and no network reload.

Phase 3. Officer portal and the 26-factor monthly audit (L4 data, L5)

MISSION

Build the Project Officer and Division Engineer experience for monthly audits, including the 26-factor structured submission defined in Appendix A.

BUILD

- Officer dashboard: assigned bridges, due and overdue audits, saved drafts, last tier per bridge, upcoming reminders.
- Multi-step wizard with five factor groups plus a review step. One schema source in `packages/shared` generates Pydantic models and Zod validators, so frontend and backend cannot drift.
- Unit-aware validation with **hard errors** (impossible values) and **soft warnings** (statistical outliers versus the bridge's last three audits, with a required justification note).
- Field help text in three languages explaining how to measure each factor, referencing IRC:SP:35 by name only.
- Evidence upload to MinIO via presigned URLs: client-side image compression, EXIF GPS check against the bridge location (warn, never block), per-factor attachments, virus-scan hook point.
- Drafts autosave to server and IndexedDB; the form tolerates network loss by queuing the submission and reconciling on reconnect, with an explicit conflict screen.
- Rules: one audit per bridge per calendar month; submitted audits are immutable (corrections via an amendment record that keeps history); late submissions flagged.
- Reviewer flow: Division Engineer approves or returns with comments; officer is notified in their language.

- On submit, enqueue a risk assessment (Phase 5) and show the officer the resulting tier and recommended actions.

ACCEPTANCE

- End-to-end test covers draft, offline submit, return-for-correction, resubmit, approve. Lighthouse accessibility 95 or higher on the wizard. Mobile layout works at 360 px width with 44 px touch targets.

Phase 4. Telemetry simulator and live monitoring (L3)

MISSION

Create a realistic, reproducible IoT simulator and the device-ready ingestion pipeline for live monitoring. Real sensors must be able to replace the simulator without changing the API.

BUILD

- Sensor types: ultrasonic water level (m), pier inclinometer (degrees), accelerometer (RMS g and dominant frequency Hz), rain gauge (mm/h), temperature. Each sensor has an ID, bridge, location on the structure, calibration metadata and a hashed API key.
- Scenario engine driven by YAML with a seeded random generator: normal monsoon cycle with diurnal and seasonal patterns, flash-flood hydrograph (rise, peak, recession), progressive scour causing slow tilt drift, seismic shaking burst, sensor faults (stuck value, spike, dropout), network outage and late-arriving batches.
- Include named replay scenarios inspired by the case studies in the report (scour-driven pier failure similar to Schoharie Creek, hill flash flood similar to Uttarakhand 2013). Label them as stylised simulations, not reconstructions.
- Ingestion API `POST /api/v1/telemetry/readings` (batch, idempotent, UTC timestamps, quality flags), plus an optional MQTT bridge using the same JSON contract. Reject unknown sensors and future timestamps beyond tolerance.
- Storage: monthly-partitioned readings table, 1-minute and 1-hour rollups, retention policy, index on (sensor, time).
- Anomaly detection: threshold, rate-of-change and EWMA z-score detectors with a sensor-health state (healthy, stale, faulty) so a dead sensor is never read as a safe bridge.
- Live dashboard over SSE: charts with threshold bands, stale-data badge, `SIMULATED` banner, last-reading age.

ACCEPTANCE

- Same seed gives identical streams. Throughput test sustains 500 readings per second locally. Injecting a flash-flood scenario visibly moves the dashboard within 5 seconds and creates anomaly events.

Phase 5. AI predictive risk engine and prescriptive remediation (L6)

MISSION

Implement the Structural Vulnerability Index, dynamic risk, Remaining Useful Life and remediation recommendations exactly as specified in Appendices A, D and the maths in Appendix D. The rule-based engine ships first and is the safety floor. Machine learning only advises.

BUILD: RULE ENGINE (v1)

- Normalisers (piecewise-linear, config-driven), group scores, SVI 0 to 100, critical-flag overrides, hazard multiplier from live telemetry and rainfall alerts, dynamic risk index and four tiers: Good, Watch, Warning, Critical.
- Reproduce the report's sample scoring (Table 7: five factors scored 1 to 3, bands 5 to 7 Low, 8 to 11 Medium, 12 to 15 High) as a named golden test so the final engine demonstrably extends the approved concept.
- Every assessment stores inputs snapshot, config hash, engine version, group scores, top-5 contributing factors, tier, RUL range and timestamp. It must be reproducible from the stored snapshot.
- Property-based tests (Hypothesis): worsening any single factor never lowers SVI; scores stay within bounds; missing data lowers confidence and is surfaced, never treated as zero risk.

BUILD: RUL AND REMEDIATION

- RUL: Markov-chain deterioration of component condition ratings (0 to 9) with transition matrices in config, outputting P50 and P10 years to the poor-condition threshold, adjusted by SVI. Mark outputs as indicative.
- Remediation catalogue (Appendix D) triggered by rules with priority, estimated urgency window, reference standard by name, and trilingual text. Output ranked actions per assessment.

BUILD: ML ADVISOR (v2)

- In `ml/`: a seeded, physics-informed synthetic data generator plus a loader for the public FHWA National Bridge Inventory to demonstrate condition-rating modelling. Document dataset limits clearly.
- Train gradient-boosted models (XGBoost with GPU on the RTX laptop) for 12-month adverse-condition probability; calibrate probabilities (isotonic), explain with SHAP, report ROC-AUC, PR-AUC, calibration curve and subgroup metrics in a model card.
- Hybrid rule: `final_tier = max(rule_tier, ml_tier)` but ML may never downgrade a rule-based critical flag. If the model is missing or unhealthy, the system falls back to rules and says so.

ACCEPTANCE

- Risk module coverage 95% or higher, golden and property tests green, assessment reproducible from snapshot, model card present, API returns explanation plus ranked recommendations in all three languages.

Phase 6. Notification engine and scheduler (L4)

MISSION

Build a reliable multi-channel notification system and the monthly reminder scheduler.

BUILD

- Celery Beat job `month_end_reminder_scan` runs daily at 08:00 IST. For each active officer-bridge assignment it sends a reminder when today equals **last day of the month minus 7 days**. Handle 28, 29, 30 and 31 day months and leap years with explicit tests. Idempotency key: officer, bridge, year-month, type.
- Escalation ladder: reminder at T-7 days, second reminder at T-2 days if no draft exists, overdue notice on the first day after month-end, escalation to the Division Engineer 3 days after that.
- Alert triggers from the risk engine and anomaly detector: tier changes (any increase), critical flags, water level crossing warning and danger levels, seismic event detected. Alerts are bridge-specific, include tier, reason, recommended action and a deep link. Deduplicate and rate-limit to prevent alert fatigue; support acknowledge, snooze (with reason) and auto-resolve.
- Channel adapters with a common interface: Email via SMTP, SMS via provider interface (mock locally; document DLT template registration for production in India), In-app via SSE. Per-user channel preferences and quiet hours that never suppress Critical alerts.
- Delivery log with status, attempts, provider response and locale used. Retries with exponential backoff and a dead-letter queue visible to admins.
- Templates in English, Hindi and Marathi for every notification type, rendered in the recipient's language.

ACCEPTANCE

- Time-travel tests (freeze time) prove reminders fire exactly 7 days before month-end across a full year including February of a leap year, never fire twice, and escalate correctly. Mailpit shows correctly rendered trilingual emails.

Phase 7. Public transparency portal, engineer console and analytics (L7)

MISSION

Deliver the citizen-facing portal, the engineers' alert console and the administrative analytics.

PUBLIC PORTAL

- Map with react-leaflet, marker clustering, tier icon plus text plus colour, search by name, district or river, and filters. Bridge page shows name, location, year built, age and share of design life used, construction quality summary, last audit date, current health tier with plain-language meaning, 12-month tier trend, and the advisory disclaimer.
- A redaction layer exposes only the safe public subset (never raw defect details, sensor credentials, exact vulnerability breakdowns or officer personal data). Visibility is controlled by a `public_visibility` flag; redaction is unit-tested.
- Citizen concern form with rate limiting and a captcha stub, routed to the Division Engineer queue.
- Performance and quality: server-side caching, Lighthouse 90 or higher on performance and 95 or higher on accessibility, small JavaScript budget, works on slow mobile networks.

ENGINEER CONSOLE AND ANALYTICS

- Alert console: acknowledge, assign, record decision and action (close, divert, inspect, repair) and closure notes, mirroring the report's decision-and-action loop.

- Analytics: state heatmap, tier distribution, overdue audits, officer compliance, alert volume, mean time to acknowledge, top recurring recommendations, CSV export.

ACCEPTANCE

- Public routes work without login, are rate-limited, expose nothing outside the safe subset (tested), and render correctly in all three languages. Playwright covers citizen, officer, engineer and admin journeys.

Phase 8. Validation, hardening and professional documentation

MISSION

Prove the system works, harden it, and document it to a standard an external reviewer would accept.

BUILD

- **Evaluation of the 90% objective:** a seeded simulation study over at least 1,000 bridge-years with randomised hazards, hidden weaknesses and sensor faults, including case-study replays. Compare a baseline (annual visual inspection plus generic regional alerts) against Setu Raksha. Report detection lead time, recall of Critical conditions, false alarms per bridge-year, and the percentage of failure scenarios in which a Critical alert and a restrict/close recommendation were raised at least 6 hours before the failure threshold. Give bootstrap confidence intervals and state the limits. Write `docs/evaluation-report.md` with plots.
- Security: STRIDE threat model, an ASVS-style checklist, ZAP baseline scan in CI, dependency and secret scanning, rate-limit and authorisation tests, upload abuse tests.
- Performance: k6 scripts; targets p95 under 300 ms for read APIs at 50 requests per second locally, and 500 telemetry readings per second sustained.
- Accessibility and i18n audit: axe on every page, keyboard-only walkthrough, native-speaker review log for Hindi and Marathi strings and the glossary.
- Documentation: C4 diagrams, API reference, data dictionary, model card, runbook (backup, restore, rotate keys, provider outage), user guides for citizen, officer and engineer in three languages, and a 10-minute demo script built around a seeded scenario called **Monsoon Night** (rising river, scour-driven tilt, alert chain, engineer closes the bridge).
- Release: tag `v1.0.0`, generate an SBOM, update CHANGELOG, and produce a one-page architecture poster and a final walkthrough video script.

ACCEPTANCE

- All checks in Appendix E pass from a clean clone; the evaluation report reproduces from one command; no high-severity findings remain open.

Appendix A. The 26-Factor Inspection Model

Save as `docs/spec/factors.md`. All thresholds are examples and live in `config/svi_thresholds.yaml`, to be finalised with a civil engineering expert. Group codes: **S** structural, **H** hydraulic, **E** seismic, **L** load, **A** age, history and environment.

ID	Factor	Input and unit	ID	Factor	Input and unit
S-01	Deck condition	Rating 0 to 9	E-01	Seat width adequacy	Provided / required ratio
S-02	Superstructure condition	Rating 0 to 9	E-02	Pier tilt	Degrees or mm per m
S-03	Substructure (piers, abutments)	Rating 0 to 9	E-03	Liquefaction susceptibility	Low / Medium / High
S-04	Bearing condition	Rating 1 to 5 plus displacement mm	E-04	Seismic zone (IS 1893)	II, III, IV, V
S-05	Maximum crack width	mm, location	L-01	Average daily traffic	Vehicles per day
S-06	Corrosion / section loss	% of section	L-02	Heavy vehicle share	%
S-07	Expansion joint condition	Rating 1 to 5	L-03	Overload incidents	Count per month
H-01	Measured scour depth	m, versus design scour	A-01	Age versus design life	Years, % used
H-02	Foundation exposure	m	A-02	Days since detailed or underwater inspection	Days
H-03	Scour protection (riprap) condition	Rating 1 to 5	A-03	Construction quality grade	A, B, C, D
H-04	Debris accumulation	None / Low / Medium / High	A-04	Open high-priority defects	Count
H-05	Water-level margin to design HFL	m (latest)	A-05	Slope / landslide susceptibility	Low / Medium / High
H-06	Channel bed level change	m since last audit	A-06	Chloride / corrosion exposure	Mild to Extreme

Live telemetry (water level, tilt, vibration, rainfall) does not add weights to SVI. It drives the hazard multiplier, anomaly flags and critical overrides in the dynamic risk index.

Appendix B. Database Entities

Entity	Key fields	Notes
organizations / jurisdictions	id, name, type (state, division, district), parent_id	Hierarchy used for scoping
users, roles, user_prefs	id, email, password_hash, role, jurisdiction_id, locale, channels, quiet_hours	Admin-created; locale drives UI and notifications
bridges	id, code, name, geom (PostGIS), river, district, type, length_m, spans, year_built, design_life_y, irc_class, seismic_zone, foundation_type, foundation_depth_m, design_hfl_m, design_scour_m, public_visibility, is_synthetic	L1 design metadata
bridge_assignments	bridge_id, officer_id, from_date, to_date	History kept; drives reminders
contractors, construction_records	contractor_id, bridge_id, quality_grade, completion_date, dlp_end, notes	L2 history
material_logs, load_tests	bridge_id, material, grade, test_value, min_required, pass, date; test_type, load, deflection_mm, result	L2 quality evidence
audits	id, bridge_id, officer_id, period (YYYY-MM), status (draft, submitted, returned, approved), submitted_at, amendment_of	One per bridge per month; immutable after submit
audit_factors	audit_id, factor_id, value_num, value_code, unit, note, lang	26 factor rows per audit; typed, not free JSON
evidence_files	id, audit_id, factor_id, object_key, mime, exif_gps, checksum	MinIO object keys
sensors, sensor_keys	id, bridge_id, type, location_on_structure, calibration, key_hash, state	Health state tracked

Entity	Key fields	Notes
telemetry_readings	sensor_id, ts (UTC), value, unit, quality, source (simulated or device)	Monthly partitions plus rollups
risk_assessments	id, bridge_id, trigger, svi, dri, tier, rul_p50, rul_p10, group_scores, top_factors, inputs_snapshot, config_hash, engine_version, model_version, confidence	Reproducible, never overwritten
recommendations, recommendation_catalogue	assessment_id, rule_id, priority, urgency, status; catalogue holds i18n keys and standard reference	Actions tracked to closure
alerts, alert_actions	id, bridge_id, severity, reason, status, ack_by, action (close, divert, inspect, repair), notes	Decision and action loop
notifications, delivery_attempts	id, user_id, type, channel, locale, status, attempts, provider_response, idempotency_key	Unique idempotency key
audit_log	id, actor, action, entity, before, after, request_id, ts	Append-only
citizen_reports	id, bridge_id, text, contact (optional), status	Rate-limited public input

Appendix C. API Surface (REST, /api/v1)

Method and path	Purpose	Access
POST /auth/login, /refresh, /logout	Session management	Any
GET /public/bridges, /public/bridges/{id}	Map list and safe bridge profile	Public
GET /public/bridges/{id}/trend	12-month tier trend	Public
POST /public/reports	Citizen concern (rate-limited)	Public
GET, POST /bridges; GET, PATCH /bridges/{id}	Registry and L1 metadata	Scoped roles
GET, POST /bridges/{id}/construction	L2 records, material logs, load tests	Engineer, Admin
POST /bridges/{id}/assignments	Assign officers	Division Engineer, Admin
GET /audits?status=&period=	List audits	Scoped roles
POST /audits; PUT /audits/{id}/draft	Create audit, autosave draft	Officer
POST /audits/{id}/submit; /amend	Submit, correct	Officer
POST /audits/{id}/approve; /return	Review	Division Engineer
POST /evidence/presign	Presigned upload URL	Officer
POST /telemetry/readings	Batch ingestion with sensor key	Sensor key
GET /bridges/{id}/telemetry?from=&to=&res=	History and rollups	Scoped roles
GET /bridges/{id}/telemetry/stream	Live SSE stream	Scoped roles
GET /bridges/{id}/risk/latest, /risk/history	SVI, tier, RUL, explanation	Scoped roles
POST /bridges/{id}/risk/recompute	Manual recompute	Engineer, Admin
GET /bridges/{id}/recommendations; PATCH /{rid}	Ranked actions, status	Scoped roles
GET /alerts; POST /alerts/{id}/ack, /snooze, /action	Alert console	Engineer
GET /notifications; PATCH /me/preferences	In-app inbox, channels, locale	Authenticated
GET /admin/analytics/*; GET /admin/jobs; GET /admin/dlq	Analytics, scheduler and queue health	Admin
GET /meta/enums?locale=	Enums with labels for forms	Any
GET /health, /ready	Liveness and readiness	Ops

Conventions: cursor pagination, ISO-8601 UTC, SI units, error body `{code, params, request_id}`, idempotency keys on submit and telemetry, rate limits on public and ingestion routes.

Appendix D. Risk Maths, Tiers and Remediation Catalogue

SPECIFICATION (agent must implement exactly; thresholds live in config)

- Each factor is normalised to a sub-score s in [0, 1] by piecewise-linear bands from `svi_thresholds.yaml`. Group score G_k is the weighted mean of its factor sub-scores.

- **SVI = 100 x (0.25 G_S + 0.25 G_H + 0.15 G_E + 0.10 G_L + 0.25 G_A).** Weights sum to 1 and are configurable.
- **Critical flags override the weighted sum** (a single severe defect must not be averaged away). Examples: measured scour at or above design scour or foundation exposure above allowance; water level at or above design HFL; pier tilt beyond threshold; bearing displaced or failed; seat width ratio below 1.0; crack width above threshold; strong seismic event in the last 72 hours near the bridge. Any flag forces a minimum tier of Warning or Critical.
- **Hazard multiplier H in [1.0, 1.5]** from live water-level ratio, 24-hour rainfall or official alert level, and recent seismic event. **DRI = min(100, SVI x H).**
- Tiers on DRI (example values): Good 0 to 29, Watch 30 to 54, Warning 55 to 74, Critical 75 to 100. Public wording: Good, Watch, Warning, Critical, each with a plain-language meaning in three languages.
- Confidence: share of the 26 factors that are present and recent. Missing or stale data lowers confidence and widens the displayed range; it never reduces risk.

Rule ID	Trigger (example)	Recommended action (catalogue key, trilingual)
RIPRAP-01	Scour ratio high or scour protection rated poor	Place or restore riprap or apron around piers; re-survey bed after next flood
SCOUR-UW-01	No underwater inspection beyond interval or scour rising	Schedule underwater inspection before next monsoon
DEBRIS-01	Debris accumulation Medium or High with rising water	Clear debris and trash racks; assign watch during high flow
BEARING-01	Bearing rated poor or displaced	Inspect and replace bearings; restrict heavy vehicles meanwhile
SEAT-01	Seat width ratio near or below 1.0 in zone IV or V	Increase seat width or install restrainers and shear keys
JACKET-01	Pier cracking with high seismic zone	Detailed assessment and column jacketing or retrofit design
CRACK-01	Crack width above threshold, no corrosion	Monitor and seal or inject; re-inspect within set window
CORR-01	Section loss or aggressive chloride exposure	Protective coating, cathodic or repair mortar; test cover and chlorides
LOAD-01	Overload incidents rising or load rating low	Enforce load limits; load-rating review
RESTRICT-01	Warning tier with flood or seismic hazard	Restrict heavy traffic and prepare diversion
CLOSE-01	Critical tier or critical flag active	Close bridge, divert traffic, dispatch inspection team, inform control room

Appendix E. Definition of Done and Recovery Prompts

E.1 Master acceptance checklist (run from a clean clone)

Area	Must be true
Run	`make up` healthy in under 5 minutes; seed creates synthetic bridges, users and a Monsoon Night scenario
Lifecycle	L1 to L7 each demonstrable end to end in the UI
Languages	All screens, errors, emails, SMS and recommendations work in English, Hindi and Marathi; language switch preserves state (Playwright proof)
Scheduler	Reminder fires exactly 7 days before month-end, once, with correct escalation (frozen-time tests across a full year)
Risk engine	Golden test reproduces report Table 7; property tests pass; assessments reproducible; critical flags override; ML cannot downgrade
Telemetry	Seeded simulator, device-ready ingestion, stale or faulty sensors never read as safe
Security	RBAC and jurisdiction tests, ZAP baseline clean, no secrets in repository, public redaction tested
Quality	CI green; coverage targets met; Lighthouse and axe thresholds met; p95 and throughput targets met
Honesty	SIMULATED labels present; advisory disclaimer on risk screens; evaluation report states limits; no real-world prevention claim
Docs	README, ADRs, C4 diagrams, API reference, data dictionary, model card, runbook, three-language user guides, demo script

E.2 Recovery prompts (paste when an agent drifts)

- **Re-plan:** Stop. Re-read `docs/spec/` and Part A rules. List where your current work deviates, propose a corrected plan as an artifact, and wait for my approval before editing more files.
- **Verify honestly:** Run lint, type-check and the full test suite now. Paste the real output. For every failure or skipped check, say why and fix it. Do not mark the task complete until everything is green.
- **i18n audit:** Search the codebase for user-facing strings outside message files and for any code path that stores translated text in the database. Fix each and add a test.
- **Scope guard:** You added features beyond this phase. Revert them into a separate branch, and list them as backlog items with rough effort.
- **Explain a decision:** Write an ADR for the choice you just made: context, options considered, decision, consequences. Link it from the README.
- **Security pass:** Review the diff as an attacker: authorisation gaps, IDOR across jurisdictions, injection, unsafe uploads, secrets, verbose errors. List findings by severity and fix high and medium ones with tests.

Appendix F. Suggested Timeline and Risks

A realistic 12-week plan for one student working with an agentic IDE. Review every agent plan and walkthrough yourself: you must be able to explain every module in your viva.

Weeks	Phase	Focus	Faculty checkpoint
1	Phase 0	Repo, Compose stack, CI, ADRs	
2 to 3	Phase 1	Schema, auth, RBAC, bridge registry, L1 and L2	
3 to 4	Phase 2	Trilingual layer, state-preserving switch, glossary	Language demo
4 to 5	Phase 3	Officer portal and 26-factor wizard	Officer workflow demo
4 to 6	Phase 4	Simulator, ingestion, live dashboard	
6 to 8	Phase 5	Rule engine, RUL, recommendations, ML advisor	Risk model review with a civil engineering faculty member
8 to 9	Phase 6	Scheduler, alerts, trilingual notifications	Month-end reminder demo
9 to 10	Phase 7	Public portal, engineer console, analytics	
10 to 12	Phase 8	Evaluation study, security, performance, docs, demo rehearsal	Final demo: Monsoon Night

F.1 Hardware notes for the RTX laptop

- Everything except model training runs in Docker on CPU. The RTX GPU is used only for XGBoost GPU training and optional experiments; the training script must fall back to CPU automatically.
- Allocate at least 16 GB RAM to Docker for the full stack. Use `make up-lite` (no simulator, no ZAP) while developing the UI.

F.2 Key risks and mitigations

Risk	Mitigation built into the prompts
Scope creep and an agent that drifts	Phase-by-phase missions, plan approval, scope-guard and re-plan prompts (Appendix E)
Unrealistic or unvalidated risk thresholds	Config-driven thresholds labelled example, expert review checkpoint, golden test tied to the approved report
Overclaiming results in the viva	Evaluation protocol with confidence intervals; SIMULATED labels; advisory disclaimers; 90% stated only as a target
Poor Hindi or Marathi technical wording	Glossary pending native-speaker review; reviewed_by field on every recommendation string
Real SMS delivery blocked in India	Provider interface with mock; DLT template registration documented as a production step
Public portal exposing exploitable detail	Redaction layer, public_visibility flag and tests; no raw defect data on public routes
Silent errors in a safety context	Sensor health states, confidence scores, missing data never treated as safe, ML cannot downgrade rule flags